

Clase 017 — Python para seguridad: manipulación de paquetes con Scapy

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Scapy Documentation / Black Hat Python* 🕒

Duración estimada: **120 min** · Nivel: **Fundamentos**

Objetivo

Construir, enviar, capturar y diseccionar paquetes de red a bajo nivel con Scapy. Al terminar podrás forjar paquetes capa por capa, implementar tu propio ping y SYN scan, esnifar tráfico y automatizar pruebas de red, uniendo todo lo aprendido sobre TCP/IP y Python.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Forjar** paquetes especificando cada capa (Ether/IP/TCP/UDP/ICMP).
2. **Enviar** y recibir paquetes con `send`, `sr`, `sr1`, `srp`.
3. **Esnifar** tráfico y aplicar filtros BPF.
4. **Implementar** un SYN scan y un descubrimiento de hosts.
5. **Diseccionar** respuestas para inferir estado y servicios.

Temas

#	Tema	Por qué importa
1	Modelo de capas en Scapy	/ apila capas
2	Construcción de paquetes	Control total de cabeceras
3	Envío y recepción	send/sr/sr1/srp
4	Sniffing	Capturar y procesar en vivo
5	Filtros BPF	Reducir ruido en la captura
6	SYN scan	Escaneo half-open a mano
7	ARP/ICMP	Descubrimiento de red
8	Análisis de respuestas	Interpretar flags y códigos

Definiciones y características

- **Scapy**: librería Python para forjar/capturar paquetes. Clave: `IPC()/TCP()` apila capas con el operador `/`.

- **sr1()**: envía un paquete y devuelve la primera respuesta. Clave: ideal para sondas puntuales (ping, un puerto).
- **srp()**: como **sr** pero a nivel de capa 2 (Ethernet). Clave: necesario para ARP.
- **Filtro BPF**: sintaxis de filtrado en captura (`tcp port 80`). Clave: evita procesar tráfico irrelevante.
- **SYN scan con Scapy**: enviar un SYN y clasificar según la respuesta (SYN-ACK=abierto, RST=cerrado). Clave: implementa a mano lo visto en la Clase 011.
- **Raw sockets**: acceso de bajo nivel que Scapy usa por debajo. Clave: requiere privilegios de root.

Herramientas y preparación

Instala Scapy en tu entorno virtual de Kali:

```
pip install scapy      # o: sudo apt install python3-scapy
sudo scapy             # consola interactiva (necesita root para enviar)
```

Trabaja siempre en la red interna del laboratorio. Ten Wireshark abierto en paralelo para verificar que tus paquetes forjados salen como esperas.

Laboratorio guiado

1. Forjar y ver un paquete:

```
python from scapy.all import * pkt = IP(dst="10.10.10.6")/ICMP() pkt.show()
```

2. Ping propio con **sr1**:

```
python resp = sr1(IP(dst="10.10.10.6")/ICMP(), timeout=2, verbose=0) print("Vivo" if resp else "Sin respuesta")
```

3. Descubrimiento ARP en la subred (capa 2):

```
python ans, _ = srp(Ether(dst="ff:ff:ff:ff:ff:ff")/ARP(pdst="10.10.10.0/24"), timeout=2, verbose=0) for _, r in ans: print(r.psrc, r.hwsrc)
```

4. SYN scan a mano de un puerto:

```
python r = sr1(IP(dst="10.10.10.6")/TCP(dport=22, flags="S"), timeout=2, verbose=0) if r and r.haslayer(TCP): print("abierto" if r[TCP].flags == 0x12 else "cerrado")
```

Envía luego un RST para cerrar limpiamente. 5. **Sniffing con filtro BPF**:

```
python pkts = sniff(filter="tcp port 80", count=10, timeout=15) pkts.summary()
```

6. Verifica en Wireshark que tus SYN forjados aparecen con los flags correctos.

 **Nota ética:** forjar y enviar paquetes se hace **solo** en tu laboratorio aislado. Inyectar tráfico en redes ajenas sin autorización es ilegal y puede causar daños.

Ejercicios

1. Modifica el ping para ajustar el TTL y observa el efecto con `tracert` manual.
2. Escribe un escáner que recorra una lista de puertos con SYN scan y clasifique cada uno.
3. Implementa un descubrimiento de hosts combinando ICMP y ARP.
4. Captura 20 paquetes y extrae con Scapy las IPs origen únicas.
5. Forja un paquete UDP a un puerto DNS y analiza la respuesta ICMP si está cerrado.
6. Añade el envío del RST tras un SYN-ACK para no dejar conexiones colgadas.

Reto verificable

Implementa `scapyscan.py`, un escáner SYN con Scapy que reciba un objetivo y una lista de puertos, clasifique cada puerto como abierto/cerrado/filtrado según la respuesta, cierre con RST los que respondan SYN-ACK, y muestre un informe. Verifica con Wireshark que los paquetes salen correctos.

Criterio de aceptación: el clasificador coincide con `nmap -sS` sobre los mismos puertos de tu VM víctima (abierto/cerrado/filtrado), y en Wireshark se observa el patrón SYN → SYN-ACK → RST para los puertos abiertos. Ejecuta con privilegios y sin dejar conexiones a medias.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
<code>PermissionError</code> / no envía nada	Scapy necesita root para raw sockets. Ejecuta con <code>sudo</code> .
Interfaz equivocada	Scapy usa la ruta por defecto. Especifica <code>iface=</code> si hace falta.
<code>sr1</code> devuelve <code>None</code> siempre	Timeout corto o host filtrado. Sube el timeout y verifica conectividad.
Los paquetes no aparecen en Wireshark	Filtros mal o miras otra interfaz. Ajusta el filtro y la NIC.
El SYN scan deja conexiones a medias	No enviaste el RST. Cierra tú la conexión tras el SYN-ACK.

Preguntas frecuentes

? ¿Para qué usar Scapy si existe nmap? Scapy da control total del paquete: puedes forjar cabeceras arbitrarias, probar comportamientos anómalos y construir herramientas a medida que nmap no cubre. Es didáctico y flexible.

? ¿Por qué necesita root? Enviar paquetes forjados requiere raw sockets, una operación privilegiada. Por eso también es una capacidad potente que debe usarse con responsabilidad.

? ¿Scapy sirve para defensa? Sí: prototipar detecciones, generar tráfico de prueba para un IDS, o analizar capturas. No es solo ofensivo.

? ¿Puedo hacer sniffing sin promiscuo? Verás tu propio tráfico, pero para capturar el de otros hosts en una LAN conmutada necesitarías port mirroring o MITM (y autorización). Recuerda el marco legal.

Referencias

- Scapy Documentation — <https://scapy.readthedocs.io/>
- Seitz & Arnold, *Black Hat Python* (cap. de Scapy).
- BPF filter syntax — <https://biot.com/capstats/bpf.html>
- Nmap vs. Scapy (comparativa didáctica) — <https://nmap.org/book/>

Siguiente clase

Clase 018 - Git y control de versiones para profesionales de seguridad